```
import pandas as pd
import seaborn as sns
#To know what are the datasets present in seaborn
print(sns.get_dataset_names())
#Load titanic Dataset
df=sns.load_dataset("titanic")
print(df.head())
print(df.shape)
df.info()
```

```
['anagrams', 'anscombe', 'attention', 'brain_networks', 'car_crashes',
'diamonds', 'dots', 'dowjones', 'exercise', 'flights', 'fmri',
'geyser', 'glue', 'healthexp', 'iris', 'mpg', 'penguins', 'planets',
'seaice', 'taxis', 'tips', 'titanic']
   survived  pclass     sex   age  sibsp  parch     fare embarked
class  \
0         0       3    male  22.0      1      0   7.2500        S
Third
1         1       1  female  38.0      1      0  71.2833        C
First
2         1       3  female  26.0      0      0   7.9250        S
Third
3         1       1  female  35.0      1      0  53.1000        S
First
4         0       3    male  35.0      0      0   8.0500        S
Third

     who  adult_male deck  embark_town alive  alone
0    man        True  NaN  Southampton    no  False
1  woman       False    C    Cherbourg   yes  False
2  woman       False  NaN  Southampton   yes   True
3  woman       False    C  Southampton   yes  False
4    man        True  NaN  Southampton    no   True
(891, 15)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   survived     891 non-null    int64
 1   pclass       891 non-null    int64
 2   sex          891 non-null    object
 3   age          714 non-null    float64
 4   sibsp        891 non-null    int64
 5   parch        891 non-null    int64
 6   fare         891 non-null    float64
 7   embarked     889 non-null    object
 8   class        891 non-null    category
 9   who          891 non-null    object
 10  adult_male   891 non-null    bool
```

```
 11  deck         203 non-null    category
 12  embark_town  889 non-null    object
 13  alive        891 non-null    object
 14  alone        891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB
```

Inconsistent format

The same value is repesentes on different format

```
#creating a problem for practice
df.loc[0,'embark_town']="southampton"
df.loc[1,'embark_town']="SOUTHAMPTON"
df.loc[2,'embark_town']="Southampton"
print(df['embark_town'].unique())
```

```
['southampton' 'SOUTHAMPTON' 'Southampton' 'Queenstown' 'Cherbourg'
nan]
```

```
#solution for the above problem
df["embark_town"]=df["embark_town"].str.title()
print(df["embark_town"].unique())
```

```
['Southampton' 'Queenstown' 'Cherbourg' nan]
```

```
#checking for numm values
print(df["embark_town"].isnull().sum())
```

```
2
```

```
df["embark_town"]=df["embark_town"].fillna("kingstown")
```

```
print(df["embark_town"].unique())
```

```
['Southampton' 'Queenstown' 'Cherbourg' 'kingstown']
```

2.Duplicate Rows

A duplicate row is an exact copy of another row

```
#creating problem to practice
duplicate_rows=df.iloc[:5]
df_dup=pd.concat([df,duplicate_rows])
print(df_dup.shape)
```

```
(896, 15)
```

```
#To find duplicates
print(df_dup.duplicated().sum())
```

```
112
```

```
#view duplicate rows
df_dup[df_dup.duplicated()]
```

```
    survived  pclass     sex   age  sibsp  parch     fare embarked
class  \
47         1       3  female   NaN      0      0   7.7500        Q
Third
76         0       3    male   NaN      0      0   7.8958        S
Third
77         0       3    male   NaN      0      0   8.0500        S
Third
87         0       3    male   NaN      0      0   8.0500        S
Third
95         0       3    male   NaN      0      0   8.0500        S
Third
..       ...     ...     ...   ...    ...    ...      ...      ...
...
0          0       3    male  22.0      1      0   7.2500        S
Third
1          1       1  female  38.0      1      0  71.2833        C
First
2          1       3  female  26.0      0      0   7.9250        S
Third
3          1       1  female  35.0      1      0  53.1000        S
First
4          0       3    male  35.0      0      0   8.0500        S
Third

      who  adult_male deck  embark_town alive  alone
47  woman       False  NaN   Queenstown   yes   True
76    man        True  NaN  Southampton    no   True
77    man        True  NaN  Southampton    no   True
87    man        True  NaN  Southampton    no   True
95    man        True  NaN  Southampton    no   True
..    ...         ...  ...          ...   ...    ...
0     man        True  NaN  Southampton    no  False
1   woman       False    C  Southampton   yes  False
2   woman       False  NaN  Southampton   yes   True
3   woman       False    C  Southampton   yes  False
4     man        True  NaN  Southampton    no   True

[112 rows x 15 columns]
```

```
df_dup=df_dup.drop_duplicates()
print(df_dup.duplicated().sum())
```

```
0
```

Typos

Spelling mistakes create unnecessary categories

```
df.loc[0,"embark_town"]="Southamtom"
df.loc[1,"embark_town"]="Southampthom"

print(df["embark_town"].value_counts())
```

```
embark_town
Southampton     643
Cherbourg       167
Queenstown       77
kingstown         2
Southampthom      1
Southamtom        1
Name: count, dtype: int64
```

```
#fix typos
df["embark_town"]=df["embark_town"].replace({
    "Southampthom":"Southampton",
    "Southamtom":"Southampton"
})
#verify
print(df["embark_town"].value_counts())
```

```
embark_town
Southampton    645
Cherbourg      167
Queenstown      77
kingstown        2
Name: count, dtype: int64
```

```
#Detect possible typos
print(sorted(df["embark_town"].dropna().unique()))
```

```
['Cherbourg', 'Queenstown', 'Southampton', 'kingstown']
```

4 Unit Mismatches

Values representing the same measurement are stored in different units

```
#Suppose some ages are acciendily entered in months so we convert into
years
df.loc[0,"age"]=240
df.loc[1,"age"]=300

#detect unusual data
df["age"].describe()
```

```
count    714.000000
mean      30.371387
std       19.362012
min        0.420000
25%       20.125000
50%       28.000000
75%       38.750000
max      300.000000
Name: age, dtype: float64
```

```
#Fix unit mismatch
#convert months into years
df.loc[df["age"]>120,"age"]=(df.loc[df["age"]>120,"age"]/12)

#verify
df["age"].describe()
```

```
count    714.000000
mean      29.678109
std       14.525902
min        0.420000
25%       20.000000
50%       28.000000
75%       38.000000
max       80.000000
Name: age, dtype: float64
```

5. Wrong Category Labels

- df["survived"]
- contains
- 0
- 1
- Not very readable

```
df["survived_label"]=df["survived"].map({
    0:"No",
    1:"Yes"
})
print(df.head())
```

```
   survived  pclass     sex   age  sibsp  parch     fare embarked
class  \
0         0       3    male  20.0      1      0   7.2500        S
Third
1         1       1  female  25.0      1      0  71.2833        C
First
2         1       3  female  26.0      0      0   7.9250        S
Third
3         1       1  female  35.0      1      0  53.1000        S
First
4         0       3    male  35.0      0      0   8.0500        S
Third

     who  adult_male deck  embark_town alive  alone survived_label
0    man        True  NaN  Southampton    no  False             No
1  woman       False    C  Southampton   yes  False            Yes
2  woman       False  NaN  Southampton   yes   True            Yes
3  woman       False    C  Southampton   yes  False            Yes
4    man        True  NaN  Southampton    no   True             No
```

Redundant Columns

Columns with little or no useful information Example:

- df["deck"]
- contains mostly missing values

```
#check
df.isnull().mean()

survived          0.000000
pclass            0.000000
sex               0.000000
age               0.198653
sibsp             0.000000
parch             0.000000
fare              0.000000
embarked          0.002245
class             0.000000
who               0.000000
adult_male        0.000000
deck              0.772166
embark_town       0.000000
alive             0.000000
alone             0.000000
survived_label    0.000000
dtype: float64
```

```
#remove
df.drop(columns=["deck"],inplace=True)
df.head()
```

```
   survived  pclass     sex   age  sibsp  parch     fare embarked
class  \
0         0       3    male  20.0      1      0   7.2500        S
Third
1         1       1  female  25.0      1      0  71.2833        C
First
2         1       3  female  26.0      0      0   7.9250        S
Third
3         1       1  female  35.0      1      0  53.1000        S
First
4         0       3    male  35.0      0      0   8.0500        S
Third

     who  adult_male  embark_town alive  alone survived_label
0    man        True  Southampton    no  False             No
1  woman       False  Southampton   yes  False            Yes
2  woman       False  Southampton   yes   True            Yes
3  woman       False  Southampton   yes  False            Yes
4    man        True  Southampton    no   True             No
```

Handling Null Values in Titanic Dataset

Null values(missing values)occur when data is unavailable,unknown values

```
#check for null values
df.isnull().sum()
```

```
survived             0
pclass               0
sex                  0
age                177
sibsp                0
parch                0
fare                 0
embarked             2
class                0
who                  0
adult_male           0
embark_town          0
alive                0
alone                0
survived_label       0
dtype: int64
```

```
#Method 1 : Mean imputation
#replace missing values with the average value
df["age"]=df["age"].fillna(df["age"].mean())
df.head(20)
```

```
    survived  pclass     sex        age  sibsp  parch     fare
embarked  \
0          0       3    male  20.000000      1      0   7.2500
S
1          1       1  female  25.000000      1      0  71.2833
C
2          1       3  female  26.000000      0      0   7.9250
S
3          1       1  female  35.000000      1      0  53.1000
S
4          0       3    male  35.000000      0      0   8.0500
S
5          0       3    male  29.678109      0      0   8.4583
Q
6          0       1    male  54.000000      0      0  51.8625
S
7          0       3    male   2.000000      3      1  21.0750
S
8          1       3  female  27.000000      0      2  11.1333
S
9          1       2  female  14.000000      1      0  30.0708
C
10         1       3  female   4.000000      1      1  16.7000
S
11         1       1  female  58.000000      0      0  26.5500
S
12         0       3    male  20.000000      0      0   8.0500
S
13         0       3    male  39.000000      1      5  31.2750
S
14         0       3  female  14.000000      0      0   7.8542
S
15         1       2  female  55.000000      0      0  16.0000
S
16         0       3    male   2.000000      4      1  29.1250
Q
17         1       2    male  29.678109      0      0  13.0000
S
18         0       3  female  31.000000      1      0  18.0000
S
19         1       3  female  29.678109      0      0   7.2250
C

     class    who  adult_male  embark_town alive  alone survived_label
```

```
0    Third    man        True  Southampton    no  False             No
1    First  woman       False  Southampton   yes  False            Yes
2    Third  woman       False  Southampton   yes   True            Yes
3    First  woman       False  Southampton   yes  False            Yes
4    Third    man        True  Southampton    no   True             No
5    Third    man        True   Queenstown    no   True             No
6    First    man        True  Southampton    no   True             No
7    Third  child       False  Southampton    no  False             No
8    Third  woman       False  Southampton   yes  False            Yes
9   Second  child       False    Cherbourg   yes  False            Yes
10   Third  child       False  Southampton   yes  False            Yes
11   First  woman       False  Southampton   yes   True            Yes
12   Third    man        True  Southampton    no   True             No
13   Third    man        True  Southampton    no  False             No
14   Third  child       False  Southampton    no   True             No
15  Second  woman       False  Southampton   yes   True            Yes
16   Third  child       False   Queenstown    no  False             No
17  Second    man        True  Southampton   yes   True            Yes
18   Third  woman       False  Southampton    no  False             No
19   Third  woman       False    Cherbourg   yes   True            Yes


#Method 2 : Median imputation
#replace missing values with the average value
df["age"]=df["age"].fillna(df["age"].median())
df.head(20)

    survived  pclass     sex        age  sibsp  parch     fare
embarked  \
0          0       3    male  20.000000      1      0   7.2500
S
1          1       1  female  25.000000      1      0  71.2833
C
```

```
2           1       3  female  26.000000      0      0   7.9250
S
3           1       1  female  35.000000      1      0  53.1000
S
4           0       3    male  35.000000      0      0   8.0500
S
5           0       3    male  29.678109      0      0   8.4583
Q
6           0       1    male  54.000000      0      0  51.8625
S
7           0       3    male   2.000000      3      1  21.0750
S
8           1       3  female  27.000000      0      2  11.1333
S
9           1       2  female  14.000000      1      0  30.0708
C
10          1       3  female   4.000000      1      1  16.7000
S
11          1       1  female  58.000000      0      0  26.5500
S
12          0       3    male  20.000000      0      0   8.0500
S
13          0       3    male  39.000000      1      5  31.2750
S
14          0       3  female  14.000000      0      0   7.8542
S
15          1       2  female  55.000000      0      0  16.0000
S
16          0       3    male   2.000000      4      1  29.1250
Q
17          1       2    male  29.678109      0      0  13.0000
S
18          0       3  female  31.000000      1      0  18.0000
S
19          1       3  female  29.678109      0      0   7.2250
C

     class    who  adult_male  embark_town alive  alone survived_label

0    Third    man        True  Southampton    no  False             No

1    First  woman       False  Southampton   yes  False            Yes

2    Third  woman       False  Southampton   yes   True            Yes

3    First  woman       False  Southampton   yes  False            Yes

4    Third    man        True  Southampton    no   True             No

5    Third    man        True   Queenstown    no   True             No
```

```
6    First    man        True  Southampton    no   True             No
7    Third  child       False  Southampton    no  False             No
8    Third  woman       False  Southampton   yes  False            Yes
9   Second  child       False    Cherbourg   yes  False            Yes
10   Third  child       False  Southampton   yes  False            Yes
11   First  woman       False  Southampton   yes   True            Yes
12   Third    man        True  Southampton    no   True             No
13   Third    man        True  Southampton    no  False             No
14   Third  child       False  Southampton    no   True             No
15  Second  woman       False  Southampton   yes   True            Yes
16   Third  child       False   Queenstown    no  False             No
17  Second    man        True  Southampton   yes   True            Yes
18   Third  woman       False  Southampton    no  False             No
19   Third  woman       False    Cherbourg   yes   True            Yes
```

```
#Method 3 : Mode imputation
#replace missing values with the average value
df["age"]=df["age"].fillna(df["age"].mode())
df.head(20)
```

```
    survived  pclass     sex        age  sibsp  parch     fare
embarked  \
0          0       3    male  20.000000      1      0   7.2500
S
1          1       1  female  25.000000      1      0  71.2833
C
2          1       3  female  26.000000      0      0   7.9250
S
3          1       1  female  35.000000      1      0  53.1000
S
4          0       3    male  35.000000      0      0   8.0500
S
5          0       3    male  29.678109      0      0   8.4583
Q
6          0       1    male  54.000000      0      0  51.8625
S
```

```
7          0       3    male   2.000000      3      1  21.0750
S
8          1       3  female  27.000000      0      2  11.1333
S
9          1       2  female  14.000000      1      0  30.0708
C
10         1       3  female   4.000000      1      1  16.7000
S
11         1       1  female  58.000000      0      0  26.5500
S
12         0       3    male  20.000000      0      0   8.0500
S
13         0       3    male  39.000000      1      5  31.2750
S
14         0       3  female  14.000000      0      0   7.8542
S
15         1       2  female  55.000000      0      0  16.0000
S
16         0       3    male   2.000000      4      1  29.1250
Q
17         1       2    male  29.678109      0      0  13.0000
S
18         0       3  female  31.000000      1      0  18.0000
S
19         1       3  female  29.678109      0      0   7.2250
C

     class    who  adult_male  embark_town alive  alone survived_label

0    Third    man        True  Southampton    no  False             No

1    First  woman       False  Southampton   yes  False            Yes

2    Third  woman       False  Southampton   yes   True            Yes

3    First  woman       False  Southampton   yes  False            Yes

4    Third    man        True  Southampton    no   True             No

5    Third    man        True   Queenstown    no   True             No

6    First    man        True  Southampton    no   True             No

7    Third  child       False  Southampton    no  False             No

8    Third  woman       False  Southampton   yes  False            Yes

9   Second  child       False    Cherbourg   yes  False            Yes

10   Third  child       False  Southampton   yes  False            Yes
```

```
11   First  woman       False  Southampton   yes   True            Yes
12   Third    man        True  Southampton    no   True             No
13   Third    man        True  Southampton    no  False             No
14   Third  child       False  Southampton    no   True             No
15  Second  woman       False  Southampton   yes   True            Yes
16   Third  child       False   Queenstown    no  False             No
17  Second    man        True  Southampton   yes   True            Yes
18   Third  woman       False  Southampton    no  False             No
19   Third  woman       False    Cherbourg   yes   True            Yes
```

- lets check age distribution
- df["age"].skew()
- if skewness is hight: use median
- if data is nearly normal: use mean

Method 4: Forward Fill(FFill)

- Use the previous value to fill missing data
- Before Imputation

```
print(df["embarked"].isnull().sum())
```

```
2
```

```
#Apply Forward Fill
df["embarked_ffill"]=df["embarked"].ffill()
#check Missing Values
print(df["embarked_ffill"].isnull().sum())
```

```
0
```

Method 5: KNN Imputation

- K-Nearest Neighbours(KNN) Imputation fills missing values using information from similar rows
- Instead of using a single value(mean/median),it finds the K most similar records and uses their values

```
import pandas as pd
import numpy as np
```

```
data={
    "age":[22,25,np.nan,24,23],
    "fare":[7.25,8.05,7.90,8.00,7.50]
}
df=pd.DataFrame(data)
print(df)
```

```
    age  fare
0  22.0  7.25
1  25.0  8.05
2   NaN  7.90
3  24.0  8.00
4  23.0  7.50
```

```
#Apply KNN Imputation
from sklearn.impute import KNNImputer
KNN=KNNImputer(n_neighbors=3)
df_KNN=pd.DataFrame(
    KNN.fit_transform(df),
    columns=df.columns)
print(df_KNN)
```

```
    age  fare
0  22.0  7.25
1  25.0  8.05
2  24.0  7.90
3  24.0  8.00
4  23.0  7.50
```

sklearn ->Scikit-learn library for machine learning and data prepocessing

impute ->Module for handling missing values

KNNImputer ->Class that fills missing values using the K-Nearest Neighbor(KNN)

Fit_transform(df) ->learns from the data and fills missing values and Converts the result back into a pandas Dataframe

```
#check Missing values
import pandas as pd
import seaborn as sns
from sklearn.impute import KNNImputer
#Load Titanic dataset
df=sns.load_dataset("titanic")
#Forward fill for embarked
df["embarked_ffill"]=df["embarked"].ffill()
#KNN Imputation for age
cols=["age","fare","sibsp","parch","pclass"]
imputer=KNNImputer(n_neighbors=5)
```

```
df[cols]=imputer.fit_transform(df[cols])
print(df.isnull().sum())
```

```
survived             0
pclass               0
sex                  0
age                  0
sibsp                0
parch                0
fare                 0
embarked             2
class                0
who                  0
adult_male           0
deck               688
embark_town          2
alive                0
alone                0
embarked_ffill       0
dtype: int64
```

```
# Backward fill
df.fillna(method="bfill",inplace=True)
or
df.bfill(inplace=True)
```

Encoding Using Titanic Dataset

Machine Learning models work with numbers not text Example sex

- male
- female
- A model cannot directly understand "male" and "female"
- We convert categories into numbers using encoding.

Types of Encoding 1.Label Encoding(Ordinal data)

- Assign a number to each category Example size
- small
- medium
- large becomes size
- 0
- 1
- 2 The order matters. When to use??
- poor<Average<good
- low<medium<high

2.One hot Encoding(Nominal data) creates a separate binary column for each category when to use?? Example

- Color
- gender
- animals
- country
- Department

3.Target Encoding Replace categories with the average value of the target variable why use target encoding?? imagine 5000 cities one-hot encoding would create: 5000 columns Not practical Target Encoding creates: 1 column

```
import pandas as pd
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
#Load Titanic dataset
df=sns.load_dataset("titanic")
#label encoding
le=LabelEncoder()
df["sex_encoded"]=le.fit_transform(df["sex"])

#One-hot Encoding
df=pd.get_dummies(
    df,columns=["embarked"]
     #drop_first=True
)

#Target Encoding
target_map=df.groupby(
    "pclass"
)["survived"].mean()
df["class_target"]=(df["pclass"].map(target_map))
print(df.head())
```

```
   survived  pclass     sex   age  sibsp  parch     fare  class    who
\
0         0       3    male  22.0      1      0   7.2500  Third    man

1         1       1  female  38.0      1      0  71.2833  First  woman

2         1       3  female  26.0      0      0   7.9250  Third  woman

3         1       1  female  35.0      1      0  53.1000  First  woman

4         0       3    male  35.0      0      0   8.0500  Third    man


   adult_male deck  embark_town alive  alone  sex_encoded  embarked_C
\
```

```
0        True  NaN  Southampton    no  False            1       False

1       False    C    Cherbourg   yes  False            0        True

2       False  NaN  Southampton   yes   True            0       False

3       False    C  Southampton   yes  False            0       False

4        True  NaN  Southampton    no   True            1       False


   embarked_Q  embarked_S  class_target
0       False        True      0.242363
1       False       False      0.629630
2       False        True      0.242363
3       False        True      0.629630
4       False        True      0.242363
```

Feature Scaling Using Titanic Dataset

Feature scaling transforms numerical features to a similar scale

why it is needed?? In the titanic dataset: Example

- Feature Range Age 0-80 ,Fare 0-512
- Here,Fare has much larger range than age
- Many ML algorithmns calculate distances,so Fare can dominate the model

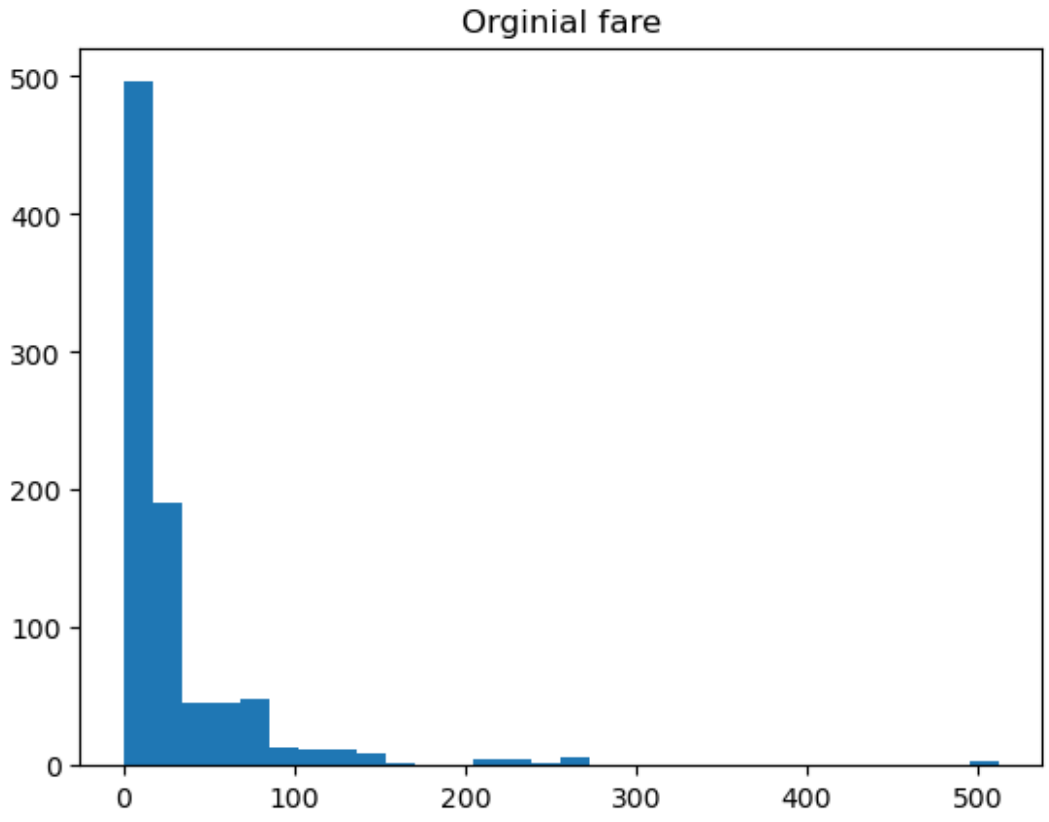
1.MinMaxScaler scales values between 0 and 1 Advantages:

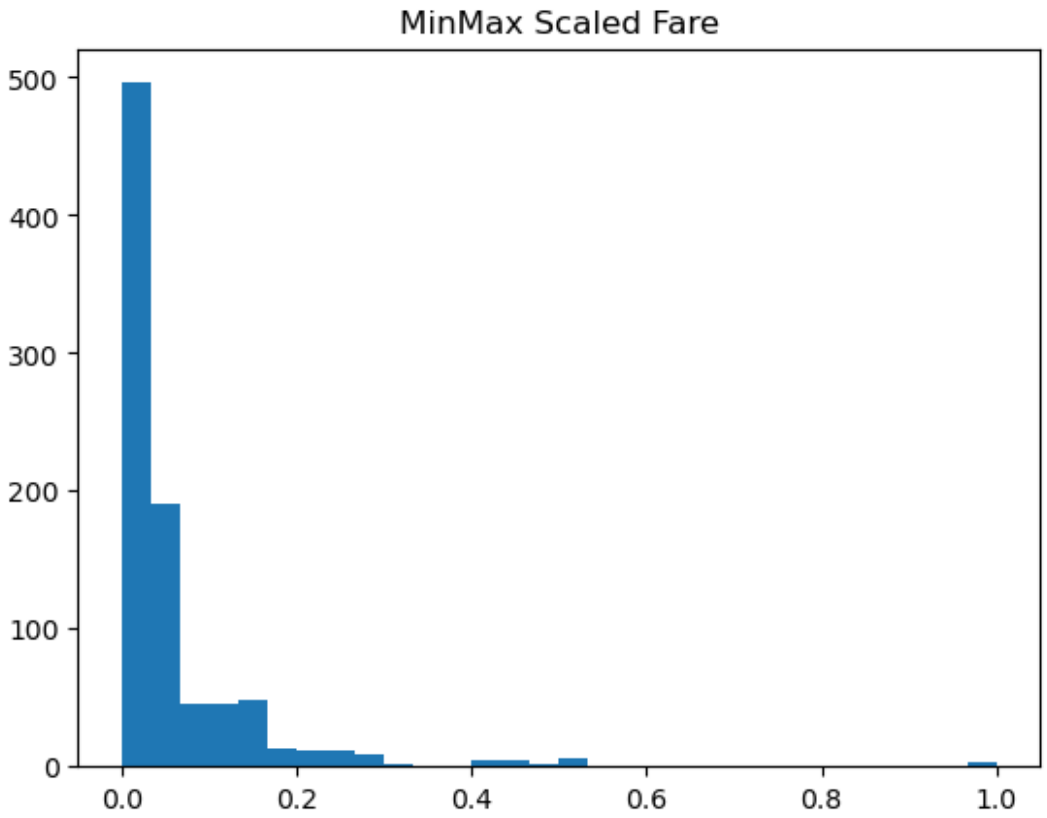
- Preserves shape of distribution
- values always between 0 and 1
- useful for neural networks disadvantage:
- sensitive to outliers 2.StandardScaler centers data around mean=0 and sitandard deviation=1 Advantage:
- Works well for normally distributed data
- commonly used
- Required by many algorithms Best for Logistic Regression,SVM,PCA,KNN,Linear Regression Disadvantage: Still affected by extreme outliers 3.RobustScaler uses median and IQR instead of mean and standard deviation advantage: handles outliers well better for skewed distributions disadvantages: Values are not limited to 0-1

```
from sklearn.preprocessing import MinMaxScaler
scaler=MinMaxScaler()
df[["age_minmax","fare_minmax"]]=scaler.fit_transform(df[["age","fare"
]])
df[["age_minmax","fare_minmax"]].head()
```

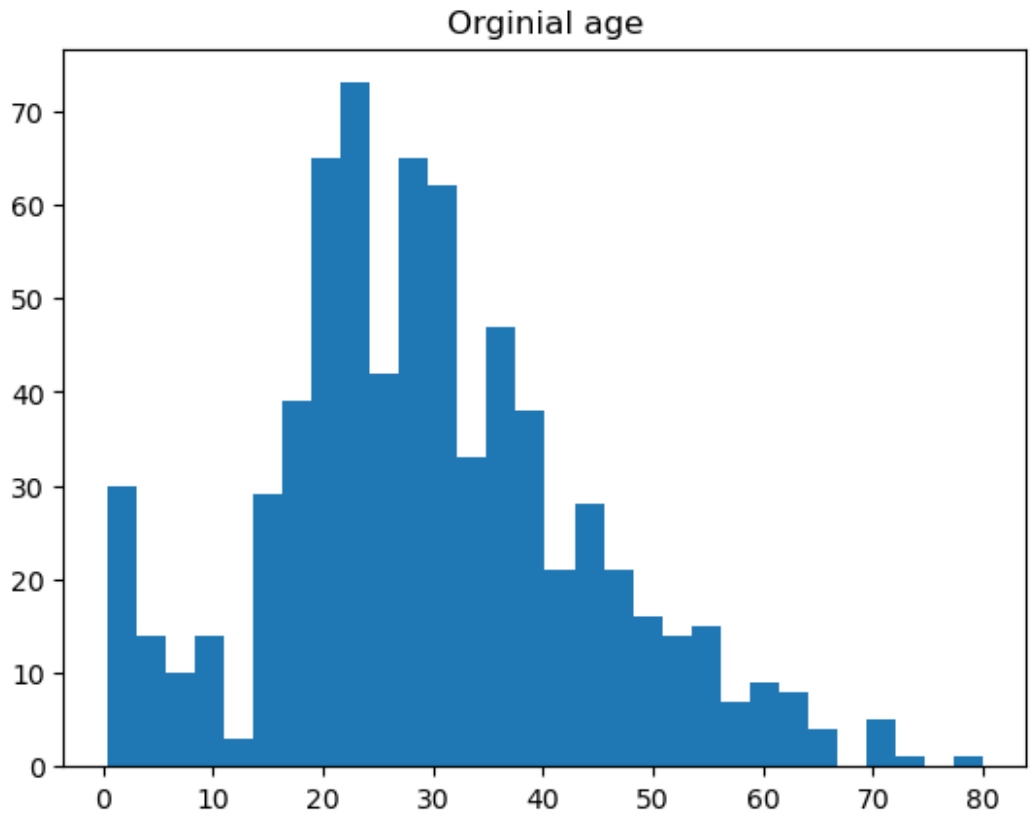
```
   age_minmax  fare_minmax
0    0.271174     0.014151
1    0.472229     0.139136
2    0.321438     0.015469
3    0.434531     0.103644
4    0.434531     0.015713
```

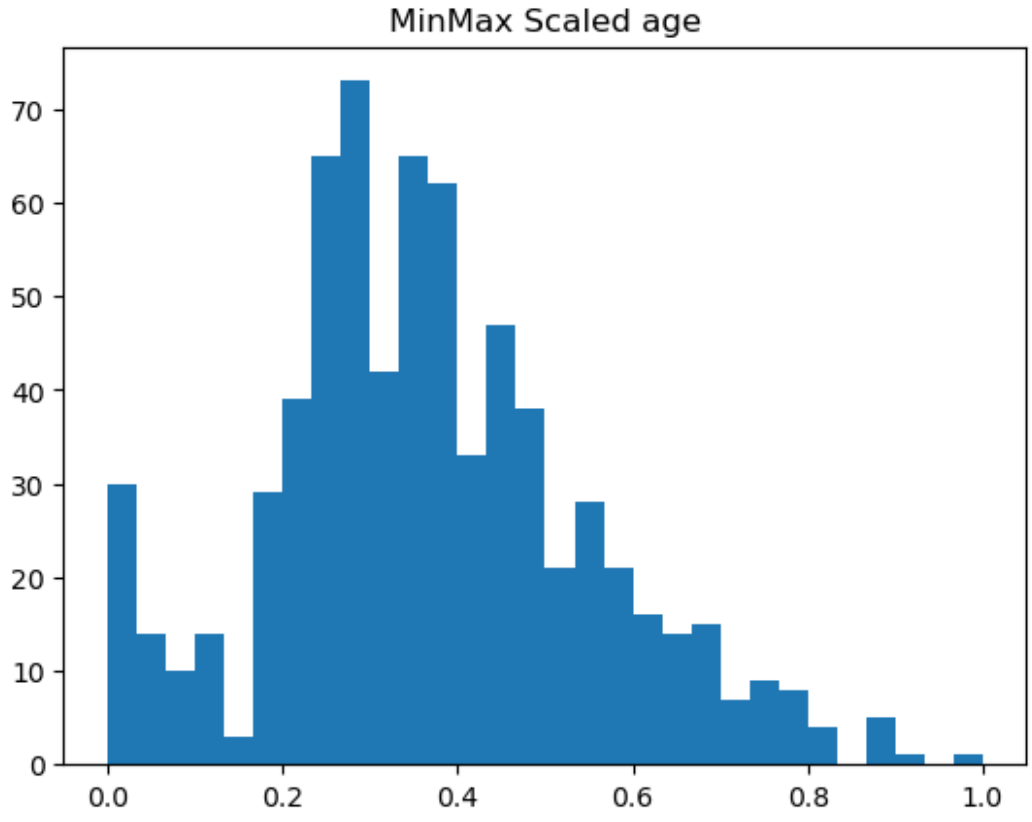
```
import matplotlib.pyplot as plt
plt.hist(df["fare"].dropna(),bins=30)
plt.title("Orginial fare")
plt.show()
plt.hist(df["fare_minmax"],bins=30)
plt.title("MinMax Scaled Fare")
plt.show()
```






```
import matplotlib.pyplot as plt
plt.hist(df["age"].dropna(),bins=30)
plt.title("Orginial age")
plt.show()
plt.hist(df["age_minmax"],bins=30)
plt.title("MinMax Scaled age")
plt.show()
```


Orginial age
70
60
50
40
30
20
10
0
0
10
20
30
40
50
60
70
80


MinMax Scaled age
70
60
50
40
30
20
10
0
0.0
0.2
0.4
0.6
0.8
1.0

```
from sklearn.preprocessing import StandardScaler
scaler=StandardScaler()
df[["age_std","fare_std"]]=scaler.fit_transform(df[["age","fare"]])
df[["age_std","fare_std"]].head()
```

```
    age_std  fare_std
0 -0.530377 -0.502445
1  0.571831  0.786845
2 -0.254825 -0.488854
3  0.365167  0.420730
4  0.365167 -0.486337
```

```
from sklearn.preprocessing import RobustScaler
scaler=StandardScaler()
df[["age_robust","fare_robust"]]=scaler.fit_transform(df[["age","fare"
]])
df[["age_robust","fare_std"]].head()
```